Il percorso del tuo primo progetto: costruiscilo tu

Versione 2.0 — 13/10/2026 — Laboratorio di Informatica, prof. Nicola Regge, Piamarta Milano.

00 Come funziona

1. L'automa che gira adesso (`automa_meteo.py`) è l'**esempio del cliente**: ti fa vedere che cosa si deve ottenere. Resta acceso, ma **non si copia**.
2. Tu costruisci il **tuo** automa da zero, nella cartella `mio-automa/`, un pezzo alla volta.
3. Ogni tappa ha: un **obiettivo**, la **prova** (che cosa devi vedere quando funziona), gli **aiuti** a tre livelli (aprili solo se ti servono, uno alla volta) e una **domanda** a cui rispondi nella chat con parole tue.
4. Il codice completo non c'è. Se ti blocchi, scrivi nella chat che cosa hai provato e che cosa vedi: ti rispondiamo con una domanda o un indizio.
5. Alla fine di ogni tappa salvi il tuo file con un commit. Claude lo controlla e ti chiede di spiegarlo. **Una tappa è fatta quando funziona e la sai spiegare.**
6. Puoi usare la chat gratuita di Claude per farti spiegare un errore o un'istruzione, non per farti scrivere la tappa.

Il tuo codice si scrive e si prova su **Google Colab** (colab.research.google.com, con l'account della scuola): ogni tappa è una cella nuova.

Tappa 1 · Il programma parte

1. **Obiettivo:** su Colab, un programma che scrive `Automa avviato` e l'ora di adesso.
2. **Prova:** sotto la cella vedi la frase e l'ora giusta di Milano.
3. **Domanda:** perché l'ora del computer di Colab può essere diversa dalla tua?

<details><summary>Aiuto 1 · l'idea</summary>

Per scrivere sullo schermo serve una funzione che stampa. Per l'ora serve un modulo che conosce date e orari, e il fuso orario di Roma.

</details>

<details><summary>Aiuto 2 · le istruzioni</summary>

<code>print()</code>, <code>from datetime import datetime</code>, <code>from zoneinfo import ZoneInfo</code>, <code>datetime.now(ZoneInfo("Europe/Rome"))</code>, <code>.strftime("%H:%M")</code>.

</details>

<details><summary>Aiuto 3 · lo schema</summary>

<pre>

adesso = datetime.now(_____)

print("Automa avviato alle", adesso.strftime(_____))

</pre>

</details>

Tappa 2 · Chiedi i dati al servizio meteo

1. **Obiettivo:** il programma chiede a Open-Meteo il meteo di Milano e stampa **tutta** la risposta.

2. **Prova:** vedi un testo lungo con parentesi graffe in cui, da qualche parte, ci sono la temperatura e il vento.
3. **Domanda:** che cos'è una API REST? Dove si trova, nella risposta, la temperatura?

L'indirizzo da chiedere (è un dato, non è codice: questo lo puoi copiare):

```
https://api.open-meteo.com/v1/forecast?latitude=45.4642&longitude=9.19&current_weather=true
```

<details><summary>Aiuto 1 · l'idea</summary>

Il programma «apre» l'indirizzo come fa il browser, legge la risposta e la trasforma da testo a dati di Python.

</details>

<details><summary>Aiuto 2 · le istruzioni</summary>

<code>import urllib.request</code>, <code>import json</code>, <code>urllib.request.urlopen(indirizzo)</code>, <code>.read()</code>, <code>.decode()</code>, <code>json.loads(testo)</code>.

</details>

<details><summary>Aiuto 3 · lo schema</summary>

<pre>

with urllib.request.urlopen(_____) as risposta:

dati = json.loads(risposta._____().decode())

print(dati)

</pre>

</details>

Tappa 3 · Prendi solo i numeri che servono

1. **Obiettivo:** una tua funzione `leggi_meteo()` che restituisce due numeri: temperatura e vento.
2. **Prova:** `print(leggi_meteo())` scrive due numeri, per esempio `(18.9, 3.8)`.
3. **Domanda:** che cosa vuol dire `dati["current_weather"]["temperature"]`? Che cosa succede se Internet non risponde, e come lo gestisci?

<details><summary>Aiuto 1 · l'idea</summary>

La risposta è come un armadio con dei cassetti: si apre il cassetto «current_weather» e dentro si prende «temperature». Una funzione raccoglie questi passi e restituisce il risultato con <code>return</code>.

</details>

<details><summary>Aiuto 2 · le istruzioni</summary>

<code>def leggi_meteo():</code>, <code>return a, b</code>, <code>try:</code> / <code>except Exception:</code>, <code>None</code>.

</details>

<details><summary>Aiuto 3 · lo schema</summary>

<pre>

def leggi_meteo():

try:

... (la tappa 2) ...

meteo = dati[_____]

return meteo[_____], meteo[_____]

except Exception:

return None, None

</pre>

</details>

Tappa 4 · Lo storico

1. **Obiettivo:** ogni lettura aggiunge una riga a `storico.csv`: data, ora, temperatura, vento. La prima riga è l'intestazione, scritta una volta sola.
2. **Prova:** scarichi il file da Colab, lo apri con Excel e vedi le colonne con **numeri veri** (Excel li allinea a destra e li sa sommare).
3. **Domanda:** perché in Excel italiano serve la virgola nei decimali? Che cosa fa la modalità «a» quando apri un file?

<details><summary>Aiuto 1 · l'idea</summary>

Aprire un file per aggiungere in fondo, senza cancellare. Controllare se il file esiste già per decidere se scrivere l'intestazione.

</details>

<details><summary>Aiuto 2 · le istruzioni</summary>

<code>open("storico.csv", "a", encoding="utf-8")</code>, <code>f.write(...)</code>, <code>import os</code>, <code>os.path.exists(...)</code>, <code>str(numero).replace(".", ",")</code>, il separatore <code>;</code>.

</details>

<details><summary>Aiuto 3 · lo schema</summary>

<pre>

def salva(data, ora, temperatura, vento):

nuovo = not os.path.exists(_____)

with open(_____, "a", encoding="utf-8") as f:

if nuovo:

f.write("Data;Ora;Temperatura;Vento\n")

f.write(f"{data};{ora};{_____};{_____}\n")

</pre>

</details>

Tappa 5 · L'automa: ripete da solo

1. **Obiettivo:** il programma fa una lettura ogni 60 secondi, finché non lo fermi.
2. **Prova:** dopo 3 minuti nello storico ci sono almeno 3 righe nuove.
3. **Domanda:** che cosa vuol dire «ciclo infinito»? Come si ferma?

<details><summary>Aiuto 1 · l'idea</summary>

Un ciclo che non finisce mai: dentro, una lettura e un salvataggio, poi una pausa.

</details>

<details><summary>Aiuto 2 · le istruzioni</summary>

<code>while True:</code>, <code>import time</code>, <code>time.sleep(60)</code>.

</details>

Tappa 6 · La dashboard

1. **Obiettivo:** il programma scrive `index.html`: il nome della tua stazione, l'ultima temperatura, il vento e la tabella delle ultime 10 letture.
2. **Prova:** scarichi il file, lo apri con doppio clic e vedi la tua pagina.
3. **Domanda:** come fa Python a scrivere una pagina web? Che cos'è una f-string?

<details><summary>Aiuto 1 · l'idea</summary>

Una pagina web è solo testo. Il programma costruisce quel testo mettendo i numeri al posto giusto e lo scrive in un file.

</details>

<details><summary>Aiuto 2 · le istruzioni</summary>

<code>f"""...{temperatura}..."""</code> (testo su più righe con i valori dentro), <code>open("index.html", "w", encoding="utf-8")</code>, i tag <code><h1></code>, <code><p></code>, <code><table></code>, <code><tr></code>, <code><td></code>, e <code><meta charset="utf-8"></code> per il «°».

</details>

Tappa 7 · Il tuo automa prende il posto dell'esempio

1. **Obiettivo:** il tuo file `mio-automa/automa.py` diventa quello vero: GitHub lo esegue ogni 30 minuti e la dashboard online è la tua.
2. **Prova:** sulla dashboard online c'è la tua pagina, e nello storico le letture automatiche.
3. **Domanda:** chi esegue il tuo programma ogni 30 minuti, e dove è scritto che lo deve fare?

<details><summary>Aiuto 1 · l'idea</summary>

Il file <code>.github/workflows/automa.yml</code> dice a GitHub quale programma eseguire e quando. Su GitHub il programma non deve girare per sempre: una lettura e basta.

</details>

Tappa 8 · Fallo crescere

Scegli tu, una alla volta: l'allarme (la pagina diventa rossa sopra una soglia), la minima, la massima e la media del giorno, un'altra città, il grafico.

Tappa 9 · La presentazione

Cinque minuti: il problema, i quattro blocchi (sorgente dati, automa, storico, dashboard), una demo dal telefono, che cosa hai imparato. Sai spiegare ogni riga del tuo codice.

Registro delle versioni

1. **v2.0 (13/10/2026):** il progetto si costruisce da zero, con gli aiuti a livelli e senza codice da copiare; l'automa di prima resta come esempio del cliente.
2. **v1.1 (13/10/2026):** come usare la chat gratuita di Claude.
3. **v1.0 (13/10/2026):** prima versione del percorso (dieci tappe con il codice già pronto).